

VibeTrader Repository Inspection Report

Purpose, architecture, workflow, dependencies, strengths, risks, and gaps

Repository	/home/user/vibetrader
Branch	add-repo-pdf-report-047492
Base branch	main
Report date	2026-05-03
Inspector	Codex task agent

Executive Summary

VibeTrader is an experimental cryptocurrency forecasting project that treats OHLCV candles as images rather than purely numeric time series. It renders 40-candle BTC/USDT or ETH/USDT windows into 256x256 candlestick charts, fine-tunes an InstructPix2Pix diffusion model to draw the next four candles, extracts a BUY/SELL/HOLD signal from the generated image, and evaluates that signal with historical backtests and a Flask-based UI.

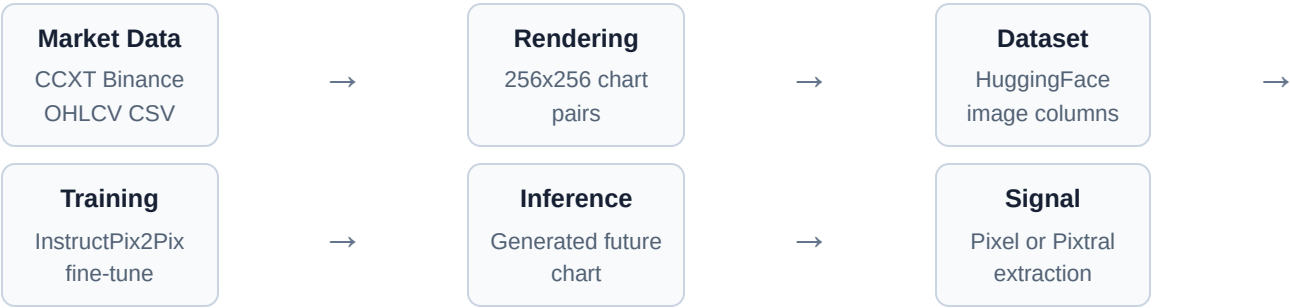
The repo is coherent as a research prototype: the data, rendering, dataset assembly, training, inference, backtesting, paper trading, and demo UI paths are all represented. The main gap is production readiness. There are no automated tests, no environment template, no committed model checkpoints or rendered training dataset, limited validation beyond two assets and one timeframe, and several code paths reference fields or files that are absent in a clean checkout.

Project Purpose

The project explores whether a diffusion image-editing model can visually continue financial candlestick charts and whether those generated continuations can drive profitable trading signals. It frames chart forecasting as conditional image generation: input image plus prompt in, edited chart with future candles and an intended signal marker out.

The README reports unusually low directional accuracy but positive simulated returns on BTC/USDT and ETH/USDT backtests. That makes the project more of a research and demonstration system than a deployable trading engine.

Architecture



The core implementation is script-oriented rather than framework-heavy. Shared behavior is mostly in Python modules under `data/` and `inference/`, with command-line entry points for rendering, dataset building, inference, backtesting, and paper trading. The web layer in `app.py` wraps the inference pipeline and serves a static HTML frontend.

Major Directories and Files

Path	Role	Notes
<code>README.md</code>	Primary project documentation	Explains the diffusion-based chart forecasting concept, quickstart, backtest summary, and limitations.
<code>data/fetch_ohlcv.py</code>	Market data ingestion	Fetches OHLCV candles via CCXT, optionally with Binance API credentials, and computes RSI/MACD via <code>pandas-ta</code> .
<code>data/render_charts.py</code>	Image pair generation	Renders 40-candle input charts and 44-candle targets with signal marker labels using PIL.
<code>data/build_dataset.py</code>	Dataset assembly	Converts rendered input/target images and prompts into a HuggingFace Dataset saved to disk.
<code>train/run_training.sh</code>	Apple Silicon/local training launcher	Finds or clones diffusers examples and launches InstructPix2Pix fine-tuning through Accelerate.
<code>train/run_training_gpu.sh</code>	Cloud GPU training launcher	Assumes <code>\$HOME/vibetrader</code> , loads <code>.env</code> , detects GPU memory, and tunes batch size.
<code>inference/predict.py</code>	Diffusion inference	Loads <code>StableDiffusionInstructPix2PixPipeline</code> from checkpoints and produces generated charts.
<code>inference/extract_signal.py</code>	Pixel-based signal extraction	Inspects the future-candle region and classifies dominant green/red pixels as BUY/SELL/HOLD.
<code>inference/extract_signal_mistral.py</code>	Vision LLM signal extraction	Uses Mistral Pixtral to interpret current and generated chart images. Requires <code>MISTRAL_API_KEY</code> .
<code>inference/generate_comparison.py</code>	Checkpoint comparison	Discovers checkpoints, generates comparison images, and writes JSON for frontend comparison views.
<code>bot/backtest.py</code>	Historical evaluation	Runs held-out-window inference, compares predicted to actual labels, and writes CSV/JSON metrics.
<code>bot/trader.py</code>	Paper trading loop	Fetches recent candles, runs prediction, and simulates long/short positioning with W&B

Path	Role	Notes
		logging.
<code>app.py</code>	Flask backend	Serves <code>frontend-v3</code> , selects showcase charts, loads the diffusion model, and exposes prediction APIs.
<code>frontend/</code> , <code>frontend-v3/</code>	Static browser UIs	HTML/CSS/JS frontends for browsing chart pairs and running interactive predictions.
<code>outputs/</code>	Generated evaluation artifacts	Contains backtest metrics/results and a small comparison image set.
<code>demo.ipynb</code> , <code>plot_pnl_distribution.py</code>	Notebook and visualization helpers	Support exploration and generation of the committed P&L distribution chart.

Dependencies

Runtime and research dependencies are listed in `requirements.txt`. The main groups are:

- Deep learning and model tooling: `torch`, `diffusers`, `transformers`, `accelerate`, `safetensors`.
- Dataset and data processing: `datasets`, `pandas`, `numpy`, `pandas-ta`, `ccxt`.
- Chart and image handling: `Pillow`, `matplotlib`, `mplfinance`.
- Experiment tracking and progress: `wandb`, `tqdm`.
- Optional vision-model extraction and UI: `mistralai`, `flask`.

`train/requirements-gpu.txt` is a slimmer GPU training dependency list and omits several runtime/UI/data-fetching packages. The training scripts also depend on HuggingFace Diffusers example scripts that are not vendored in the repository unless cloned during setup.

Operational Workflow

1. Install Python dependencies in a virtual environment with `pip install -r requirements.txt`.
2. Fetch Binance candles with `python data/fetch_ohlcv.py --symbol BTC/USDT --timeframe 4h --days 730`.
3. Render image pairs with `python data/render_charts.py --csv data/btc_usdt_4h.csv --output data/rendered`.
4. Build a HuggingFace dataset with `python data/build_dataset.py --rendered data/rendered --output data/dataset`.
5. Train with `bash train/run_training.sh` or `bash train/run_training_gpu.sh`.
6. Run backtests with `python -m bot.backtest --csv ... --checkpoint checkpoints/ --output outputs/backtest --max-samples 250`.

7. Launch the demo UI with `python app.py`, which loads model checkpoints and serves prediction routes on port 8923.

Operational caveat: a clean checkout contains CSV samples and backtest outputs, but not the rendered training dataset under `data/rendered/` or model checkpoints under `checkpoints/`. The interactive UI and inference commands require those artifacts to be generated or restored first.

Observed Backtest Artifacts

Run	Accuracy	Strategy Return	Buy & Hold	Samples
BTC/USDT 100-sample	35.3%	+40.84%	-22.57%	102
BTC/USDT 250-sample	27.5%	+107.97%	-50.09%	255
ETH/USDT 250-sample	25.1%	+95.98%	-60.04%	251

These numbers come from committed files in `outputs/backtest*/backtest_metrics.json` and `outputs/BACKTEST_REPORT.md`. They are useful research artifacts, but they are not proof of live profitability because fees, slippage, market-impact, risk constraints, and broader regime validation are absent.

Strengths

- Clear, novel research direction with a consistent end-to-end implementation.
- Simple chart renderer produces deterministic input/target image pairs and metadata prompts.
- Inference is cleanly separated from signal extraction, allowing pixel-based and vision-LLM interpretations.
- Backtesting writes both per-sample CSVs and summary JSON, making later analysis straightforward.
- The README honestly documents important limitations and provides a usable quickstart.
- Static frontends make it easy to inspect examples without adding a heavy JavaScript build system.

Risks

Financial/model risk: Backtests cover a narrow asset/timeframe/regime slice, omit fees and slippage, and use a small sample count. Reported returns should be treated as exploratory, not tradable.

- No automated test suite exists for rendering, signal extraction, backtesting math, API routes, or training setup.
- Important generated artifacts are absent from the checkout: `checkpoints/`, `data/rendered/`, and `data/dataset`.
- Several runtime paths will fail in a clean environment unless artifact prerequisites and credentials are provided.
- `bot/trader.py` and `inference/generate_comparison.py` reference `signal.avg_rgb`, but `Signal` in `extract_signal.py` has no such field.

- The Flask app performs expensive model loading and warmup at process start and has no health endpoint, auth, job queue, or resource limits.
- The Mistral integration manually parses `.env` and sends chart images to a third-party API; operational privacy and cost controls are not documented.
- Training scripts assume external Diffusers example code and W&B state; reproducibility depends on mutable upstream code unless pinned more tightly.

Notable Gaps

- Add a `.env.example` documenting optional `BINANCE_API_KEY`, `BINANCE_SECRET`, `MISTRAL_API_KEY`, W&B settings, and checkpoint/data paths.
- Add smoke tests for `compute_signal`, chart rendering dimensions, pixel signal extraction thresholds, and backtest return calculations.
- Add artifact management instructions for checkpoints and rendered datasets, including expected directory layouts and file sizes.
- Compute and report risk metrics such as max drawdown, Sharpe/Sortino, trade count, exposure, fees, slippage, and confidence-conditioned returns.
- Validate across more assets, exchanges, timeframes, and market regimes with walk-forward or out-of-sample evaluation.
- Fix stale references to `avg_rgb` or extend the `Signal` dataclass if that metric is still needed.
- Make UI startup degrade gracefully when checkpoints or rendered showcase images are missing.

Conclusion

VibeTrader is best understood as a compact research prototype for diffusion-based visual market forecasting. It contains the ingredients needed to reproduce an experiment and inspect its outputs, but it needs stronger reproducibility, tests, artifact handling, risk analysis, and operational guardrails before it can be considered reliable trading infrastructure.